Búsqueda Local con el Algoritmo Hill Climbing (Ascenso de Colina)

Fundamentos, variantes, patologías y demostración práctica:
escape de un laberinto con trampas

Asignatura: Fundamentos de Inteligencia Artificial / Algoritmos de Optimización
Equipo: [Nombres de los integrantes]
Profesor/a: [Nombre]
Fecha: [dd/mm/aaaa]

1. Introducción

En Inteligencia Artificial, muchos problemas no se resuelven encontrando un *camino* desde un estado inicial hasta una meta, sino encontrando un *estado* que sea bueno según algún criterio: el acomodo de N reinas sin ataques, la ruta más corta de un viajante o los parámetros que maximizan el rendimiento de un modelo. En estos casos el camino recorrido es irrelevante y lo único que importa es la calidad del estado final. Los algoritmos de **búsqueda local** aprovechan esa propiedad: mantienen únicamente un estado actual y lo mejoran iterativamente moviéndose a estados vecinos, por lo que usan memoria constante y pueden encontrar soluciones aceptables en espacios enormes o continuos donde la búsqueda sistemática es inviable (Russell y Norvig, 2020).

Hill Climbing es el algoritmo de búsqueda local más sencillo: en cada iteración se desplaza al vecino que mejora la función de evaluación y se detiene cuando ninguno la mejora. Su analogía es la de un alpinista con niebla espesa que solo percibe el terreno inmediato a sus pies. Es rápido y simple, pero es *miope*: puede quedar atrapado en máximos locales, mesetas o crestas.

Este reporte documenta (i) el marco teórico del algoritmo, sus variantes y sus patologías, y (ii) una demostración en **Python** que resuelve el **escape de un laberinto con trampas cóncavas**, mostrando paso a paso con telemetría tanto un caso de éxito como un caso de atrapamiento en un máximo local, y evaluando dos técnicas de mitigación: *reinicios aleatorios* y *penalizaciones*.

2. Marco teórico

2.1 Búsqueda local frente a búsqueda informada global

Los algoritmos como A* o Dijkstra exploran el espacio de forma *sistemática*: guardan en memoria una frontera de nodos (un árbol o grafo de búsqueda) y los expanden en orden de costo. Así garantizan completitud y, con una heurística admisible, optimalidad del camino, pero su memoria crece como $O(b^d)$ en el peor caso. La búsqueda local, en cambio, conserva solo el estado actual (y evalúa temporalmente a sus vecinos), con lo que la memoria es **O(1)** respecto al tamaño del espacio.

Criterio	A* / Dijkstra (global)	Hill Climbing (local)
Qué almacena	Frontera + conjunto de explorados (árbol/grafo)	Solo el estado actual
Memoria	$O(b^d)$ en el peor caso	O(1)
Resultado	Camino de costo mínimo (con heurística admisible)	Un estado; el camino no importa
Completitud / optimalidad	Sí (grafos finitos / h admisible)	No: puede detenerse en un óptimo local
Uso típico	Navegación, planificación de rutas	Optimización: N-reinas, TSP, ajuste de parámetros

2.2 Componentes formales

• **Espacio de estados S:** conjunto de configuraciones posibles. Un estado $s \in S$ representa una solución candidata completa; la solución buscada es $s^* \in S$ con la mejor aptitud.

• **Operador de vecindad $N(s) \subseteq S$:** reglas que generan los estados adyacentes a s mediante una pequeña modificación (mover una pieza, intercambiar dos aristas, desplazarse una celda).

• **Función de evaluación $f: S \to \mathbb{R}$:** cuantifica la calidad. En *maximización* se busca $s^* = \text{argmax } f(s)$; un problema de *minimización* de un costo c(s) se convierte en maximización con $f(s) = -c(s)$. Se define $\Delta f(n) = f(n) - f(s)$ para $n \in N(s)$.

Un estado s es un **máximo local** si $\forall\, n \in N(s)$: $f(n) \leq f(s)$; es un **máximo global** si $\forall\, s' \in S$: $f(s') \leq f(s)$. Todo máximo global es local, pero no al revés; Hill Climbing solo puede garantizar llegar a un máximo *local*.

2.3 Pseudocódigo

```
SIMPLE HILL CLIMBING
s ← estado_inicial
repetir:
  mejoró ← falso
  para cada n en N(s) (orden fijo):
     si f(n) > f(s):
        s ← n ; mejoró ← verdadero
        romper
  si no mejoró: devolver s
```

```
STEEPEST-ASCENT HILL CLIMBING
 s ← estado_inicial
 repetir:
   V ← N(s)
   n* ← argmax_{n ∈ V} f(n)
   si f(n*) ≤ f(s):
      devolver s   # máx. local
   s ← n*
```

La diferencia clave: Simple Hill Climbing acepta el *primer* vecino que mejora (evalúa pocos vecinos por iteración), mientras que Steepest-Ascent evalúa *todo* el vecindario y toma el de mayor pendiente (más costo por iteración, pasos de mejor calidad).

2.4 Estudio comparativo de variantes

Variante	Selección del vecino	Ventaja	Desventaja
Simple	Primer vecino mejor en orden fijo	Muy barata por iteración	Sesgada por el orden; mismo problema de óptimos locales
Steepest-Ascent	Mejor vecino (máx. Δf)	Menos pasos hasta el óptimo local	Evalúa todo N(s): costosa si |N(s)| es grande
Stochastic	Aleatoria entre los mejores, P(n) = Δf(n) / Σ Δf(m)	Menos determinista; explora distintas rutas de ascenso	Convergencia más lenta; sigue sin escapar de máximos locales
First-Choice	Vecinos generados al azar; el primero que mejora	Ideal cuando |N(s)| es enorme	Puede requerir muchas pruebas cerca del óptimo
Random-Restart	Repite el algoritmo desde estados aleatorios	Completo con probabilidad 1 si hay tiempo ilimitado	Costo total depende de la tasa de éxito p de cada corrida

Análisis de Random-Restart. Si cada corrida alcanza el óptimo global con probabilidad p, el número esperado de corridas es 1/p y el de *reinicios* es (1 − p)/p. Con las corridas de Steepest-Ascent del caso B (p = 0.588, Sección 4) se predicen 0.70 reinicios en promedio, y se midieron 0.68: la teoría concuerda con el experimento.

2.5 Patologías del paisaje de búsqueda

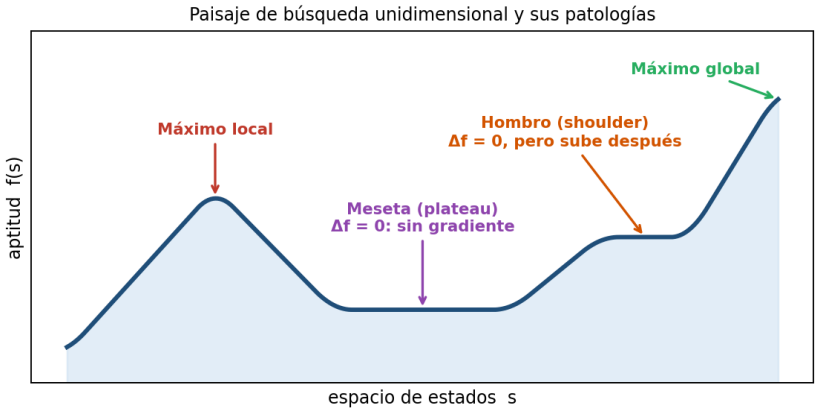


Figura 1. Paisaje unidimensional: máximo local, meseta, hombro y máximo global.

• **Máximo local:** ∃ s ≠ s* tal que ∀ n ∈ N(s): Δf(n) ≤ 0 pero f(s) < f(s*). El algoritmo termina con una solución subóptima. En el laberinto ocurre cuando los movimientos que acercan a la meta están bloqueados por muros.

• **Meseta (plateau) y hombro (shoulder):** región donde Δf = 0 para todos los vecinos (∇f ≈ 0). Sin gradiente el algoritmo no distingue direcciones. Si la meseta es un *hombro* tiene una salida ascendente; si es un máximo plano no. Se mitiga con *movimientos laterales* limitados: en el problema de las 8 reinas, Steepest-Ascent resuelve ≈14 % de las instancias, y permitir hasta 100 movimientos laterales sube el éxito a ≈94 % (Russell y Norvig, 2020).

• **Cresta (ridge):** secuencia de máximos locales cuya dirección de ascenso es diagonal respecto a los movimientos permitidos. Para f(x, y) = x + y − 5|x − y| el punto (4, 4) tiene f = 8 y sus cuatro vecinos ortogonales tienen f = 4 o f = 2; el algoritmo se detiene aunque f crece a lo largo de la diagonal (Figura 2).

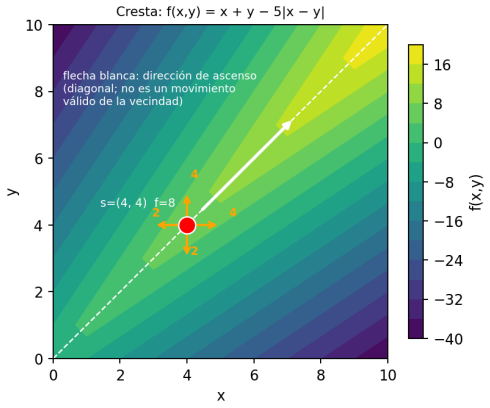


Figura 2. Cresta: movimientos ortogonales no pueden subir por la diagonal.

3. Modelado del problema implementado: escape de un laberinto con trampas

Se eligió el problema recomendado: navegación en una cuadrícula 2D con muros que forman trampas cóncavas. Se implementó en Python 3 (matplotlib para la visualización).

3.1 Definición formal del estado

Un laberinto es una cuadrícula de n × m celdas con un conjunto de muros W, una celda inicial s_0 y una meta g. Un **estado** es la celda ocupada por el agente: s = (r, c) con $0 \le r < n$, $0 \le c < m$ y $(r, c) \notin W$. El espacio de estados es S = {celdas libres}. La **solución** es el estado s = g.

3.2 Generación del vecindario

N(s) = { (r + Δr, c + Δc) : (Δr, Δc) ∈ {(−1,0), (0,1), (1,0), (0,−1)} } ∩ S, es decir, las celdas libres adyacentes en cuatro direcciones (arriba, derecha, abajo, izquierda), sin atravesar muros ni salir del tablero. El orden fijo anterior se usa en Simple y Steepest para desempates deterministas.

3.3 Función de aptitud

Se **maximiza** el negativo de la distancia a la meta:

$$f(s) = -h(s, g), \quad h_{Manhattan} = |r - r_g| + |c - c_g|, \quad h_{Euclidiana} = \sqrt{((r - r_g)^2 + (c - c_g)^2)}$$

El óptimo global es f(g) = 0. Con distancia Manhattan, cada movimiento cambia f exactamente en ±1, por lo que en este modelo no hay mesetas: toda celda sin vecino mejor es un máximo local. Una celda s ≠ g es máximo local si y solo si todas las direcciones que reducen h (hacia la meta en filas y columnas) están bloqueadas por muros o por el borde. Una **trampa cóncava** (U) con el fondo hacia la meta genera exactamente esta situación (Figura 3).

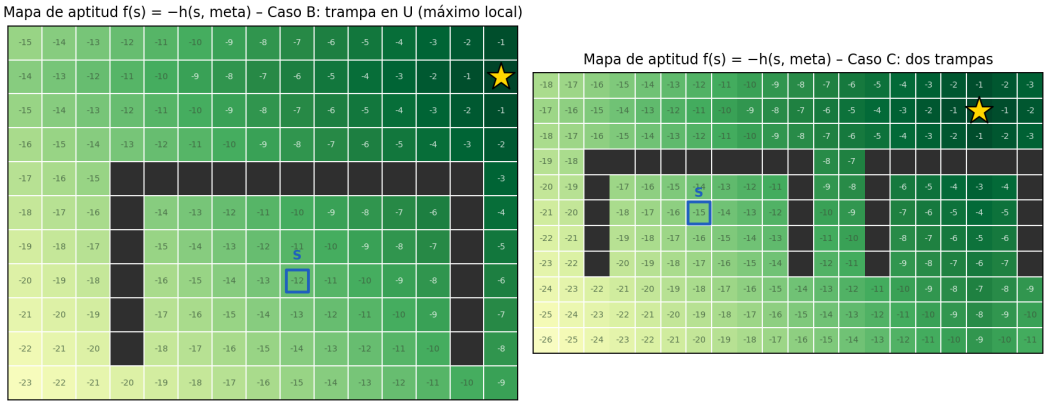


Figura 3. Mapa de aptitud f(s) en los casos B y C: los muros crean cuencas donde subir lleva a un callejón sin salida.

3.4 Casos de prueba y mitigaciones

Caso	Descripción	Resultado esperado
A	Cuadrícula 11×15, obstáculos convexos aislados	Convergencia al óptimo global
B	Una copa en U (11×15); el inicio está dentro de ella	Atrapamiento en el máximo local (5, 12), f = −6
C	Dos copas en U (11×20); el inicio está dentro de la izquierda	Atrapamiento; sirve para medir tasas de éxito

• **Reinicios aleatorios:** al terminar en un máximo local se elige una celda libre al azar como nuevo estado inicial (hasta 20 reinicios).

• **Penalizaciones:** se optimiza f′(s) = f(s) − λ·v(s), donde v(s) cuenta cuántas veces se penalizó la celda s (λ = 1). Cuando ningún vecino mejora f′, en lugar de detenerse se incrementa v(s) y se reevalúa: el máximo local se «rellena» progresivamente hasta que algún vecino resulta mejor y el agente puede salir de la cuenca. No requiere teletransportarse a otro estado.

3.5 Arquitectura del software

`hc/problem.py` (estados, vecindad y f), `hc/algorithms.py` (variantes como *generadores* que emiten un objeto *Step* por iteración), `hc/visual.py` (demo interactiva), `main.py` (consola paso a paso), `benchmark.py` (métricas). Cada *Step* contiene la telemetría exigida: estado actual, f(actual), lista de vecinos con f(vecino) y Δf, vecino elegido y razón de la decisión o de terminación. La interfaz tiene botones *Paso*, *Auto* y *Reiniciar*, y selectores de caso y algoritmo (Figura 4).

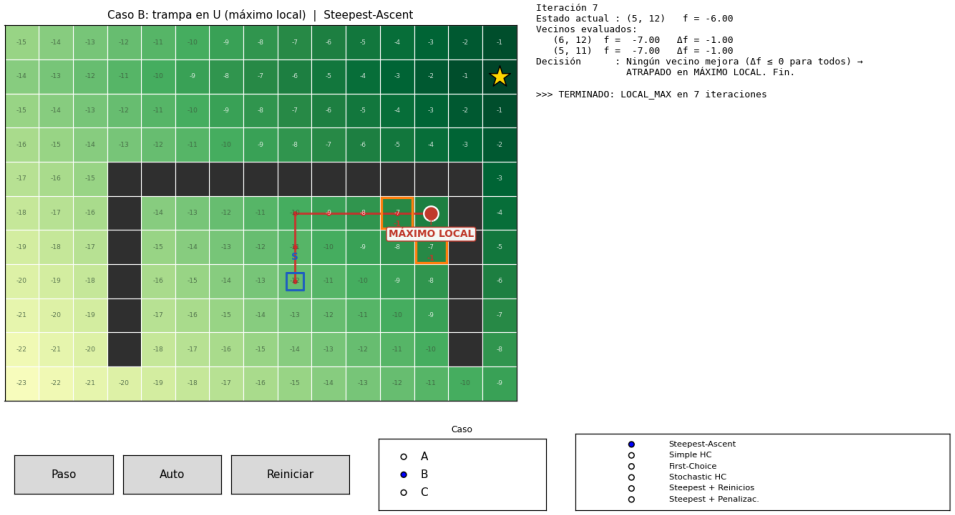


Figura 4. Interfaz de la demostración: modo paso a paso con telemetría (caso B, Steepest-Ascent, iteración 7).

4. Análisis de resultados

4.1 Comparación del algoritmo simple frente a las trampas

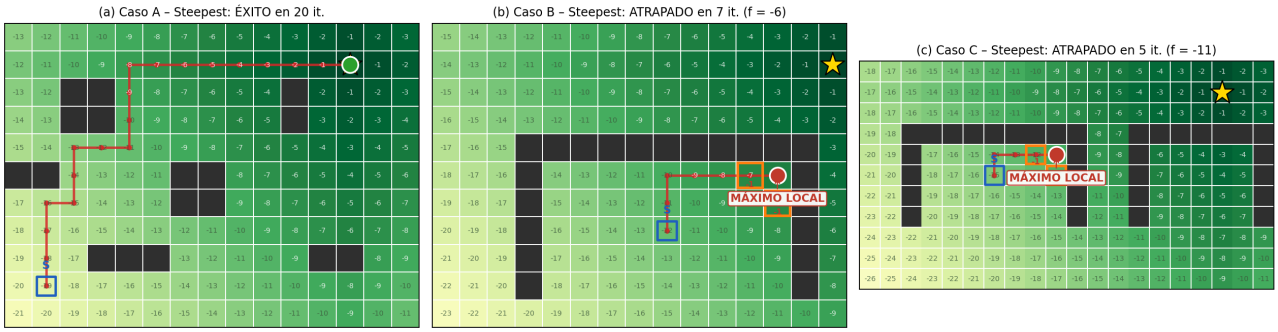


Figura 5. Steepest-Ascent desde el inicio fijo: éxito en el caso A (a) y atrapamiento en los máximos locales de los casos B (b) y C (c). Los recuadros naranja/verde son los vecinos evaluados con su Δf; ningún vecino tiene Δf > 0.

En el caso A, el algoritmo avanza con Δf = +1 en cada paso rodeando los obstáculos y llega a la meta en 20 iteraciones. En el caso B, las iteraciones 1 a 6 son ascensos legítimos, pero en la iteración 7 el agente llega a (5, 12) con f = −6: su vecino superior y el derecho son muros y los otros dos tienen Δf = −1. La meta está a solo 6 celdas, pero alcanzarla exige *empeorar* temporalmente (bajar a la salida de la copa, donde f baja hasta −11), algo que Hill Climbing nunca hace.

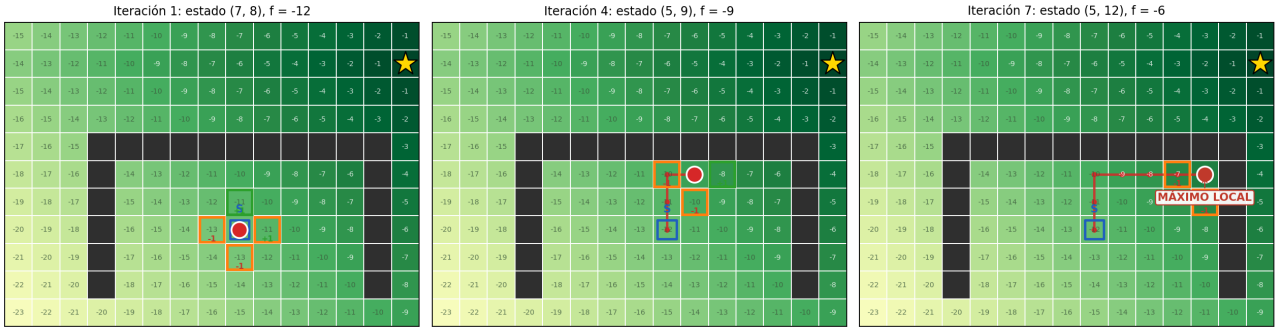


Figura 6. Caso B: iteraciones 1, 4 y 7 (atrapamiento).

4.2 Resultados con mitigaciones (inicio fijo)

Caso	Algoritmo	Resultado	Iter.	Reinicios	Estado final	f final
A	Steepest-Ascent	Éxito (óptimo global)	20	0	(1, 12)	0
A	Steepest + Reinicios aleatorios	Éxito (óptimo global)	20	0	(1, 12)	0
A	Steepest + Penalizaciones	Éxito (óptimo global)	20	0	(1, 12)	0
B	Steepest-Ascent	Atrapado en máximo local	7	0	(5, 12)	-6
B	Steepest + Reinicios aleatorios	Éxito (óptimo global)	18	1	(1, 14)	0
B	Steepest + Penalizaciones	Éxito (óptimo global)	153	0	(1, 14)	0
C	Steepest-Ascent	Atrapado en máximo local	5	0	(4, 9)	-11
C	Steepest + Reinicios aleatorios	Éxito (óptimo global)	9	1	(1, 17)	0
C	Steepest + Penalizaciones	Éxito (óptimo global)	96	0	(1, 17)	0

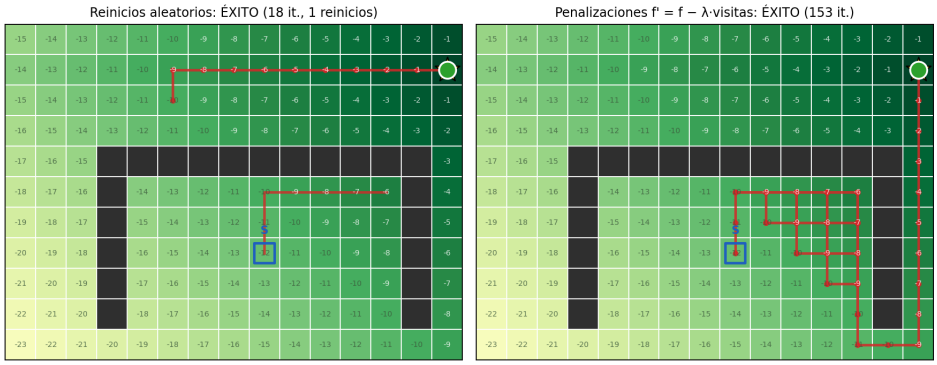


Figura 7. Caso B con mitigaciones: tras un reinicio aleatorio (izq.) y con penalizaciones (der.), donde el agente «rellena» la cuenca hasta poder salir.

4.3 Tabla de métricas (N = 500 estados iniciales aleatorios por caso)

Tasa de éxito = % de corridas que alcanzan la meta; estancamiento = % que terminan en un máximo local; las iteraciones y el tiempo son promedios por corrida (en las variantes con reinicios incluyen todos los reinicios). El tiempo incluye generar la telemetría, por lo que solo debe usarse para comparar entre algoritmos.

Caso A (obstáculos convexos)

Algoritmo	Éxito (%)	Estancado (%)	Iter. medias	Reinicios medios	Tiempo (ms)
Simple HC	97.0	3.0	10.6	0.00	0.056
Steepest-Ascent	97.0	3.0	10.6	0.00	0.089
First-Choice	94.2	5.8	10.4	0.00	0.080
Stochastic HC	94.4	5.6	10.4	0.00	0.092
Steepest + Reinicios aleatorios	100.0	0.0	10.9	0.03	0.111
First-Choice + Reinicios aleatorios	100.0	0.0	10.9	0.06	0.071
Steepest + Penalizaciones	100.0	0.0	10.9	0.00	0.087

Caso B (una trampa en U)

Algoritmo	Éxito (%)	Estancado (%)	Iter. medias	Reinicios medios	Tiempo (ms)
Simple HC	58.8	41.2	10.0	0.00	0.067
Steepest-Ascent	58.8	41.2	10.0	0.00	0.093
First-Choice	59.4	40.6	10.1	0.00	0.082
Stochastic HC	59.0	41.0	10.1	0.00	0.120
Steepest + Reinicios aleatorios	100.0	0.0	16.7	0.68	0.157
First-Choice + Reinicios aleatorios	100.0	0.0	17.3	0.71	0.160
Steepest + Penalizaciones	100.0	0.0	70.2	0.00	0.729

Caso C (dos trampas)

Algoritmo	Éxito (%)	Estancado (%)	Iter. medias	Reinicios medios	Tiempo (ms)
Simple HC	50.2	49.8	9.5	0.00	0.051
Steepest-Ascent	50.2	49.8	9.5	0.00	0.103
First-Choice	47.2	52.8	9.5	0.00	0.075
Stochastic HC	47.4	52.6	9.6	0.00	0.086
Steepest + Reinicios aleatorios	100.0	0.0	19.0	0.96	0.177
First-Choice + Reinicios aleatorios	100.0	0.0	20.1	1.10	0.231
Steepest + Penalizaciones	100.0	0.0	197.8	0.00	2.322

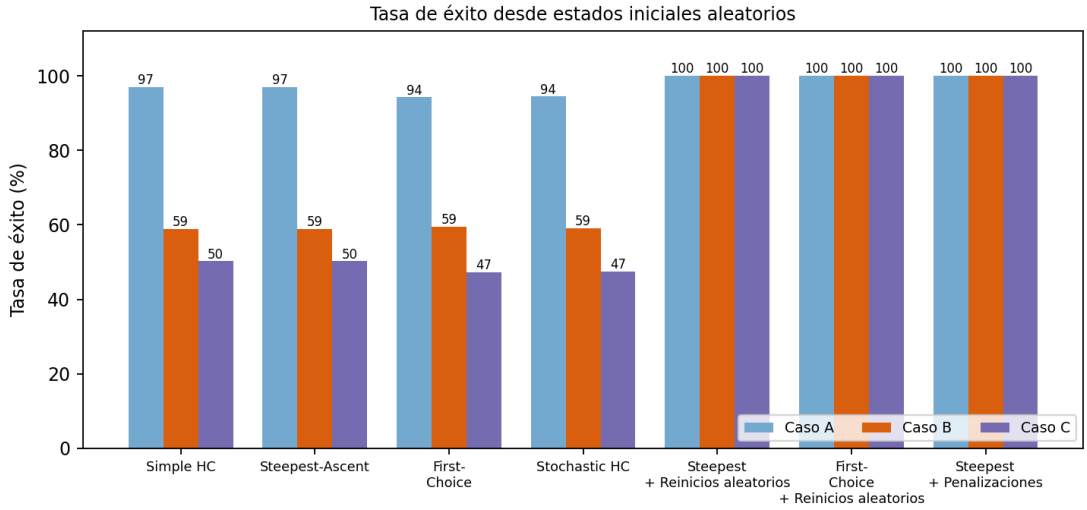


Figura 8. Tasa de éxito por algoritmo y caso.

Observaciones.

• Sin mitigación, la tasa de éxito depende del *inicio*: ≈59 % en el caso B y ≈50 % en el C; el resto de las corridas termina atrapado en una de las cuencas.

• Simple y Steepest alcanzan la misma tasa de éxito porque, con Manhattan, todo movimiento que mejora vale +1; difieren en el trabajo por iteración (Simple evalúa menos vecinos). First-Choice y Stochastic cambian la *ruta* de ascenso, lo que puede llevar a cuencas distintas, pero no eliminan los máximos locales.

• **Reinicios aleatorios** llevan el éxito a 100 % con un costo moderado (≈1.7× las iteraciones en el caso B), porque cada reinicio es un nuevo intento independiente con probabilidad p de éxito.

• **Penalizaciones** también alcanzan 100 % y no necesitan acceso aleatorio al espacio de estados, pero cuestan más iteraciones (≈7× en B y ≈20× en C) porque deben rellenar la cuenca antes de salir; su costo crece con la profundidad y el área de la trampa.

• Incluso en el caso A, ≈3–6 % de los inicios aleatorios quedan atrapados en las concavidades que forman los bordes de los obstáculos: los máximos locales aparecen aunque no se diseñen trampas.

5. Conclusiones y discusión

Idoneidad en el mundo real. Hill Climbing es la elección adecuada cuando (i) el espacio es enorme y solo se necesita una solución buena, no la óptima, (ii) la memoria es limitada, (iii) el paisaje es poco rugoso o casi unimodal, o (iv) se usa como *refinamiento local* final dentro de otro método. Es muy rápido, fácil de implementar y no requiere almacenar el espacio. Su principal debilidad, demostrada en los casos B y C, es la dependencia total del estado inicial y de la geometría local: sin garantías de optimalidad, puede fallar en ≈40–50 % de los inicios incluso en un problema tan pequeño.

Cuándo sustituirlo:

Situación	Alternativa	Justificación
Muchos máximos locales; se acepta cierto costo computacional	Simulated Annealing	Acepta movimientos que empeoran con probabilidad $e^{\Delta f/T}$; al enfriar T converge. Escapa de las trampas sin reiniciar (Kirkpatrick et al., 1983).
Paisaje deceptivo o multimodal, soluciones combinables	Algoritmos Genéticos	Mantienen una *población*; el cruce combina buenas partes y la mutación explora (Holland, 1975).
Se requiere el camino óptimo garantizado (navegación real)	A* / Dijkstra	Completos y óptimos; el costo de memoria es aceptable en mapas moderados.
Mesetas extensas	HC con movimientos laterales, Tabú o Guided Local Search	Permiten recorrer zonas planas y penalizan lo ya visitado (Voudouris y Tsang, 1999).

Aprendizajes. (1) Que la heurística sea buena no basta: Manhattan es admisible, pero como función de *aptitud local* engaña en presencia de muros. (2) La mitigación tiene un costo: los reinicios cuestan iteraciones extras y las penalizaciones aún más, pero ambas convierten un algoritmo incompleto en uno que converge con alta probabilidad. (3) La telemetría paso a paso (vecinos, Δf y razón de decisión) hace visible el momento exacto en que se produce el atrapamiento: $\forall\, n \in N(s),\ \Delta f \leq 0$. **Limitaciones:** los laberintos son pequeños y los tiempos medidos (< 3 ms) son del orden de la sobrecarga de Python; las conclusiones sobre el *costo relativo* son más fiables que los valores absolutos.

6. Referencias bibliográficas

Russell, S. y Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4.ª ed.). Pearson. (Cap. 4: Search in Complex Environments.)

Hoos, H. H. y Stützle, T. (2004). *Stochastic Local Search: Foundations and Applications*. Morgan Kaufmann.

Kirkpatrick, S., Gelatt, C. D. y Vecchi, M. P. (1983). Optimization by Simulated Annealing. *Science*, 220(4598), 671–680.

Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

Voudouris, C. y Tsang, E. (1999). Guided local search and its application to the traveling salesman problem. *European Journal of Operational Research*, 113(2), 469–499.

Hart, P. E., Nilsson, N. J. y Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.

Anexo: reproducción de resultados

```
pip install -r requirements.txt
python main.py                              # demo interactiva (Paso / Auto / Reiniciar)
python main.py --cli --case B               # consola, ENTER = siguiente paso
python main.py --cli --case B --mitigation restart --auto
python benchmark.py -n 500                  # tabla de métricas (semilla fija)
python make_figures.py && python make_report.py
```